Arduino verstehen: Schritt für Schritt

Stand: Oktober 2026 · Version 63

Dieses Lernheft führt dich vom ersten Blinken bis zum Aufbau mit Sensoren. Zu jedem Kapitel gibt es **Probier es aus**-Knöpfe: Sie öffnen das passende Beispiel direkt im CodeLab. Mit den **Aufgaben**-Knöpfen startest du eine Übung mit automatischer Prüfung.

Inhalt

1. Was ist ein Arduino?
2. Aufbau eines Programms
3. Digitale Ausgänge: LED ansteuern
4. Digitale Eingänge: Taster abfragen
5. Analoge Eingänge: Spannung messen
6. PWM: Helligkeit und Drehzahl
7. Der Serial-Monitor: dem Arduino zuhören
8. Schleifen, Entscheidungen und Felder
9. Sensoren und Aktoren
10. Fehler finden
11. Vom Simulator zum echten Uno

1. Was ist ein Arduino?

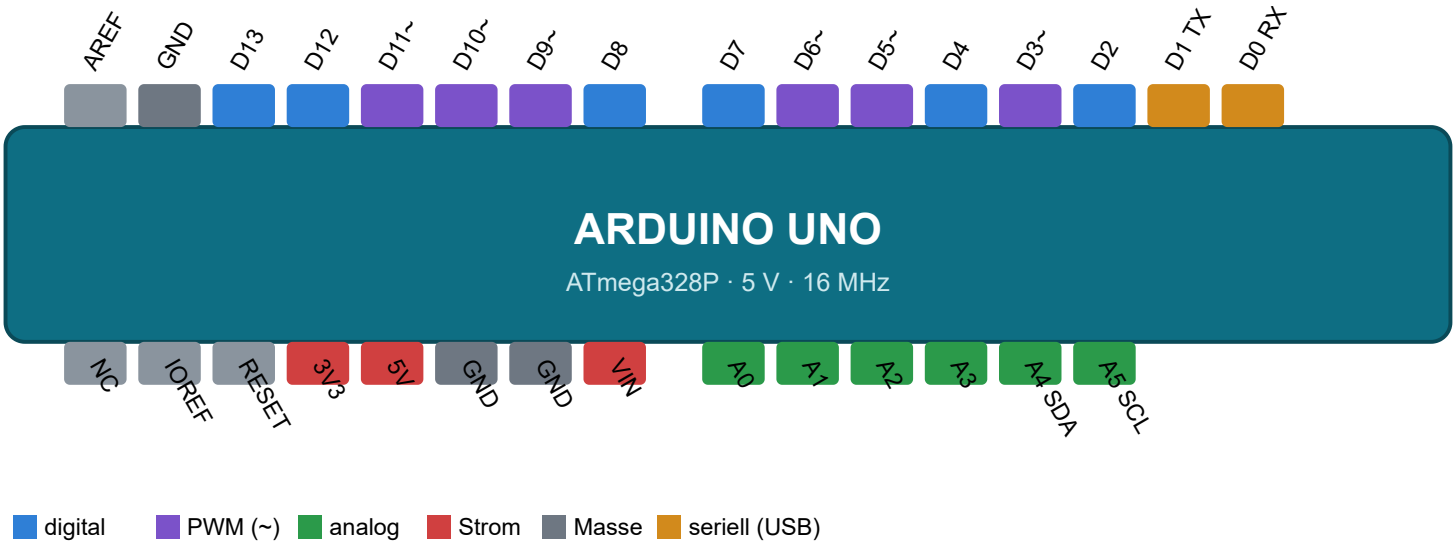
Ein **Arduino** ist ein kleiner Computer auf einer Platine, ein sogenannter **Mikrocontroller**. Er führt genau ein Programm aus, das du schreibst, und das sofort nach dem Einschalten startet. Er hat keinen Bildschirm und kein Betriebssystem. Dafür hat er **Anschlüsse (Pins)**, an die du LEDs, Taster, Sensoren und Motoren hängst.

Der **Arduino Uno** ist das bekannteste Modell:

Merkmal	Wert beim Uno
Mikrocontroller	ATmega328P
Betriebsspannung der Pins	5 V
Takt	16 MHz (16 Millionen Schritte pro Sekunde)
Programmspeicher (Flash)	32 KB (dein Programm)
Arbeitsspeicher (RAM)	nur 2 KB (deine Variablen)
Digitale Pins	14 (D0 bis D13), davon 6 mit PWM
Analoge Eingänge	6 (A0 bis A5)
Strom pro Pin	empfohlen bis 20 mA, absolutes Maximum 40 mA

> **Merke:** Der Arduino kann nur wenig Strom liefern. Eine LED braucht deshalb immer einen **Vorwiderstand**. Motoren und Lampen schaltet man über einen Transistor oder ein Relais.

Die Anschlüsse im Überblick



Oben die digitalen Pins, unten Stromversorgung und analoge Eingänge. Pins mit „~" können PWM (Kapitel 6). D0 und D1 sind mit der USB-Schnittstelle verbunden, nutze sie am besten nicht. An D13 sitzt die eingebaute LED.

▶ Im CodeLab öffnen: Blink

2. Aufbau eines Programms

Ein Arduino-Programm (man sagt **Sketch**) besteht immer aus zwei Teilen:

```
void setup() {
  // läuft EINMAL beim Start: Pins einstellen
}

void loop() {
  // läuft IMMER WIEDER, solange der Arduino Strom hat
}
```

- Jede Anweisung endet mit einem **Semikolon** `;`.
- Was zusammengehört, steht in **geschweiften Klammern** `{ }`.
- Mit `//` beginnt ein **Kommentar**. Der Arduino ignoriert ihn, er ist für Menschen.
- Groß- und Kleinschreibung zählt: `digitalWrite` ist nicht `digitalwrite`.

Variablen

Eine Variable ist ein Behälter mit Namen für einen Wert.

```
int ledPin = 13;        // ganze Zahl (-32768 bis 32767)
float spannung = 3.3;   // Kommazahl
```

```
const int taster = 2;   // Konstante: ändert sich nie
bool an = true;         // wahr oder falsch
```

> **Merke:** Gib Pins sprechende Namen (`ledPin` statt einfach `13`). Dann kannst du später nur an einer Stelle umstecken.

▶ Blink (LED an Pin 13) ▶ SOS mit eigenen Funktionen

3. Digitale Ausgänge: LED ansteuern

Ein digitaler Pin kennt nur zwei Zustände: **HIGH** (5 V) und **LOW** (0 V). Als Ausgang schaltet er damit eine LED ein oder aus.

```
pinMode(13, OUTPUT);      // Pin 13 ist ein Ausgang (einmal in setup)
digitalWrite(13, HIGH);   // 5 V, LED an
delay(1000);              // 1000 Millisekunden warten
digitalWrite(13, LOW);    // 0 V, LED aus
```

Der Vorwiderstand

Eine LED begrenzt ihren Strom nicht selbst. Der Widerstand übernimmt das. Die Rechnung:

$R = (U_{Quelle} - U_{LED}) / I$

Beispiel: rote LED (etwa 2 V), 5 V vom Arduino, gewünscht 15 mA. R = (5 V − 2 V) / 0,015 A = **200 Ω**. Nimm den nächsten Normwert: **220 Ω**.

> **Achtung:** Ohne Vorwiderstand fließt viel zu viel Strom. Im Simulator wird der Pin dann zerstört (Rauch und Meldung). Beim echten Arduino kann der Pin dauerhaft kaputtgehen.

Die LED hat eine Richtung: Das längere Bein (Anode, +) kommt zur Quelle, das kürzere (Kathode, −, abgeflachte Seite) zur Masse.

▶ Blink (LED an Pin 13) ▶ Ampel mit drei LEDs ✎ Aufgabe: Dreimal blinken

✎ Aufgabe: Die LED bleibt dunkel

4. Digitale Eingänge: Taster abfragen

Ein Pin kann auch **lesen**: Liegt 5 V an, liefert `digitalRead` den Wert HIGH, bei 0 V den Wert LOW. Ein offener Eingang „schwebt" und liefert zufällige Werte. Darum braucht jeder Taster einen festen Ruhepegel.

Der interne Pull-up

Der Arduino hat in jedem Pin einen zuschaltbaren Widerstand nach 5 V. Mit `INPUT_PULLUP` liegt der Pin im Ruhezustand auf HIGH. Der Taster verbindet ihn beim Drücken mit Masse: **gedrückt = LOW**.

```
pinMode(2, INPUT_PULLUP);     // Taster zwischen Pin 2 und Masse
if (digitalRead(2) == LOW) {   // LOW = gedrückt!
  digitalWrite(13, HIGH);
} else {
  digitalWrite(13, LOW);
}
```

> **Merke:** Mit Pull-up ist das Signal umgekehrt. Das verwirrt am Anfang: Gedrückt heißt LOW.

Prellen: Echte Taster federn kurz nach (einige Millisekunden) und liefern mehrere Impulse. Für Zähler brauchst du am echten Arduino eine kurze Wartezeit (`delay(20)`) nach dem Erkennen. Der Simulator prellt nicht.

▶ Taster schaltet LED (Pull-up) | ✎ Aufgabe: Taster schaltet die LED | ✎ Aufgabe: Tastendrücke zählen | ✎ Aufgabe: Der Taster arbeitet verkehrt herum

5. Analoge Eingänge: Spannung messen

An den Pins A0 bis A5 kann der Arduino Spannungen von 0 bis 5 V **messen**. Ein Analog-Digital-Wandler (ADC) macht daraus eine Zahl von **0 bis 1023** (10 Bit, 1024 Stufen). Eine Stufe sind 5 V / 1023 ≈ 4,9 mV.

```
int wert = analogRead(A0);                // 0 ... 1023
float spannung = wert * 5.0 / 1023.0;     // in Volt umrechnen
```

Typische Quellen: ein **Poti** (Spannungsteiler: außen 5 V und Masse, der Schleifer geht zum Pin), ein Fotowiderstand mit festem Widerstand oder ein Temperatursensor.

Werte umrechnen mit map()

```
int prozent = map(wert, 0, 1023, 0, 100);   // 0...1023 -> 0...100
```

Beispiel Temperatursensor TMP36: Er liefert 0,5 V bei 0 °C und 10 mV pro Grad. Also `celsius = (spannung - 0.5) * 100.0`.

> **Merke:** Rechne mit Kommazahlen (`5.0`), sonst teilt der Arduino ganzzahlig und rundet ab. `5 / 1023` ergibt 0!

▶ Spannung messen | ▶ Poti-Schwelle | ▶ Temperatur mit dem TMP36 | ▶ Nachtlicht | ✎ Aufgabe: Poti-Schwelle | ✎ Aufgabe: Temperaturwarnung

6. PWM: Helligkeit und Drehzahl

Ein Arduino-Pin kann keine Zwischenspannung ausgeben. Mit **PWM** (Pulsweitenmodulation) schaltet er sehr schnell zwischen 0 V und 5 V hin und her (beim Uno etwa 490 Mal pro Sekunde). Das Verhältnis von „an" zu „aus" (das **Tastverhältnis**) bestimmt, wie hell eine LED wirkt oder wie schnell ein Motor läuft.

```
analogWrite(9, 0);     //   0 % -> immer aus
analogWrite(9, 128);   //  50 % -> halbe Helligkeit
analogWrite(9, 255);   // 100 % -> immer an
```

`analogWrite` gibt es nur an den Pins mit „~": 3, 5, 6, 9, 10 und 11. Der Wert geht von 0 bis 255. Passt der Messwert eines Potis (0 bis 1023) nicht dazu, teile durch 4 oder nutze `map`.

▶ Poti dimmt LED (PWM) ▶ RGB-Farbwechsel ✎ Aufgabe: Halbe Helligkeit (PWM)

✎ Aufgabe: Poti dimmt die LED

7. Der Serial-Monitor: dem Arduino zuhören

Über die USB-Verbindung kann der Arduino Text an den Computer schicken. Das ist dein wichtigstes Werkzeug zum **Fehlersuchen**: Du siehst, was das Programm gerade denkt.

```
void setup() {
  Serial.begin(9600);          // Geschwindigkeit: 9600 Bit pro Sekunde
}
void loop() {
  int wert = analogRead(A0);
  Serial.print("Messwert: ");  // ohne Zeilenumbruch
  Serial.println(wert);        // mit Zeilenumbruch
  delay(200);
}
```

- Im CodeLab siehst du die Ausgabe unten im **Serial-Monitor** (Karte „Text"). Die Karte „Plotter" zeichnet Zahlen als Kurve.
- Du kannst auch Zeichen **an den Arduino senden** (Eingabefeld unten) und mit `Serial.read()` lesen.
- Zu viel Ausgabe bremst das Programm. Nimm bei schnellen Schleifen ein `delay`.

▶ Spannung messen ▶ LED per Serial ✎ Aufgabe: Hallo im Serial-Monitor ✎ Aufgabe: Zählen bis 10

8. Schleifen, Entscheidungen und Felder

if / else: Entscheidungen

```
if (wert > 511) {
  digitalWrite(13, HIGH);
} else {
  digitalWrite(13, LOW);
}
```

Vergleiche: == gleich (zwei Gleichheitszeichen!), != ungleich, < > <= >=. Verknüpfen: && (und), || (oder), ! (nicht).

for: eine bestimmte Anzahl wiederholen

```
for (int i = 0; i < 3; i++) {   // i = 0, 1, 2
  digitalWrite(13, HIGH);
  delay(300);
  digitalWrite(13, LOW);
  delay(300);
}
```

Felder (Arrays)

Ein Feld speichert mehrere Werte unter einem Namen. Gezählt wird ab **0**.

```
int ledPins[] = {3, 5, 7, 9};
for (int i = 0; i < 4; i++) {
  pinMode(ledPins[i], OUTPUT);
}
```

Eigene Funktionen

```
void blinkKurz() {
  digitalWrite(13, HIGH); delay(100);
  digitalWrite(13, LOW);  delay(100);
}
```

> **Merke:** `if (x = 5)` mit nur einem Gleichheitszeichen weist zu, statt zu vergleichen. Das ist einer der häufigsten Fehler.

▶ Lauflicht | ▶ Binärzähler | ✎ Aufgabe: Lauflicht | ✎ Aufgabe: Binärzähler | ✎ Aufgabe: Zählt falsch

9. Sensoren und Aktoren

Ultraschall HC-SR04: Abstand messen

Der Sensor sendet einen Schallimpuls und misst die Zeit, bis das Echo zurückkommt. Der Pin Trig startet die Messung (10 µs HIGH), der Pin Echo bleibt so lange HIGH, wie der Schall unterwegs ist. Schall braucht etwa 58 µs

für 1 cm Entfernung (hin und zurück). Also: **Abstand in cm = Zeit in µs / 58**. Mit `pulseIn(echoPin, HIGH)` misst du diese Zeit.

▶ Ultraschall-Abstandsmesser ▶ Einparkhilfe ✎ Aufgabe: Abstandswarner

Servo: Winkel einstellen

Ein Modellbau-Servo fährt einen Winkel von 0 bis 180° an. Er bekommt alle 20 ms einen Impuls, dessen Länge den Winkel bestimmt (1 ms = 0°, 1,5 ms = 90°, 2 ms = 180°). Die Bibliothek `Servo.h` erledigt das. Ein großer Servo braucht eine eigene Stromversorgung, der Arduino liefert nur wenig Strom.

```
#include <Servo.h>
Servo meinServo;
void setup() { meinServo.attach(9); }     // Signalpin
void loop()  { meinServo.write(90); }     // Winkel in Grad
```

▶ Servo Sweep ▶ Servo mit Poti ✎ Aufgabe: Servo auf 90 Grad ✎ Aufgabe: Servo fährt hin und her ✎ Aufgabe: Der Servo rührt sich nicht

Lautsprecher und Summer: Töne

```
tone(8, 440);          // Ton mit 440 Hz auf Pin 8 (Kammerton a)
delay(500);
noTone(8);             // Ton aus
```

Je höher die Frequenz, desto höher der Ton. Ein 100-Ω-Widerstand in Reihe schützt den Pin.

▶ Melodie ✎ Aufgabe: Kammerton a ✎ Aufgabe: Sirene

Bewegungsmelder (PIR)

Der PIR-Sensor erkennt die Wärmestrahlung bewegter Personen. Sein Ausgang geht bei Bewegung auf HIGH. Echte Sensoren brauchen nach dem Einschalten 30 bis 60 Sekunden zum Einmessen.

▶ Bewegungsmelder

Klopf- und Berührungssensor

Ein Piezo-Element erzeugt beim Klopfen eine kleine Spannung, die du mit `analogRead` misst. Die Berührungsfläche nutzt die Bibliothek `CapacitiveSensor`.

▶ Klopfsensor ▶ Touch-Lampe

Display (LCD)

Ein LCD mit I²C-Adapter braucht nur vier Leitungen: Plus, Masse, SDA (A4) und SCL (A5). Mit der Bibliothek `LiquidCrystal_I2C` schreibst du Text.

▶ LCD mit

10. Fehler finden

Fehlermeldungen beim Start

Wenn ▶ Start eine Fehlermeldung zeigt, steht dort die **Zeilennummer**. Schau dir genau diese Zeile an, oft auch die Zeile davor.

Meldung / Symptom	Meist die Ursache
Syntaxfehler, „erwartet ;“	Semikolon am Zeilenende vergessen
„nicht deklariert“	Variable oder Funktion falsch geschrieben (Groß/Klein!) oder nicht angelegt
Klammer fehlt	Zu jeder { gehört eine }, zu jeder (eine)
LED leuchtet nie	`pinMode(…, OUTPUT)` vergessen, falscher Pin, LED falsch herum, kein Vorwiderstand nach Masse
Taster wirkt umgekehrt	Pull-up: gedrückt = LOW
Zahlen komisch (Division ergibt 0)	Ganzzahl-Division: `5.0` statt `5` schreiben
Servo/Sensor tut nichts	`attach`/`pinMode` fehlt, Pins vertauscht, Versorgung nicht angeschlossen

Vorgehen bei Logikfehlern

1. Eingrenzen: Funktioniert der Aufbau? Ein Beispielprogramm ausprobieren.
2. Mit `Serial.println` Zwischenwerte ausgeben.
3. Ein Stück Programm auskommentieren (`//`) und prüfen, ob der Fehler verschwindet.
4. Nur **eine** Änderung auf einmal machen.

✎ Aufgabe: Die LED bleibt dunkel ✎ Aufgabe: Der Taster arbeitet verkehrt herum

✎ Aufgabe: Fehlermeldung lesen ✎ Aufgabe: Der Servo rührt sich nicht ✎ Aufgabe: Zählt falsch

11. Vom Simulator zum echten Uno

Was der Simulator genau macht (und was nicht)

- Er rechnet Schaltung und Programm gemeinsam. Gut für: Logik, Beschaltung, Vorwiderstände, Zeitabläufe.

- Er ist **idealer** als die Wirklichkeit: Taster prellen nicht, Messwerte rauschen nicht, Bauteile haben keine Toleranzen.
- Er kennt keine Speichergrenzen. Auf dem echten Uno hast du nur 2 KB RAM.
- Nicht alle Bibliotheken gibt es. Unterstützt sind Servo, LiquidCrystal(_I²C), CapacitiveSensor und die gängigen Funktionen.

Programm auf den echten Arduino bringen

1. Im CodeLab `⤓ .ino` (Datei sichern) oder `Kopieren` wählen.
2. Auf einem **Computer** die kostenlose **Arduino IDE** öffnen (arduino.cc), Programm einfügen.
3. Uno per USB-Kabel anschließen (ein Datenkabel, kein reines Ladekabel!). Bei Nachbauten ist manchmal ein CH340-Treiber nötig.
4. Werkzeuge → Board: **Arduino Uno**, Werkzeuge → Port: den neuen Port wählen.
5. Mit „Hochladen" (Pfeil) übertragen. Das Programm startet sofort.

Am iPad: Das iPad kann einen Uno nicht direkt programmieren (Safari bietet keinen Zugriff auf die USB-Schnittstelle). Du kannst die ganze Planung und das Testen am iPad machen und die .ino-Datei dann auf einen Computer in der Werkstatt übertragen (AirDrop, E-Mail, Teams, Schulcloud).

Aufbau auf dem Lochraster oder Steckbrett

1. Erst die Stromlaufplan im CodeLab zeichnen und simulieren.
2. Mit dem Ausbilder aus dem Plan ein **Layout** für die Lochrasterplatine erzeugen (LochLab, Beispiele „Arduino").
3. Beim Verdrahten Masse zuerst, Strom zuletzt. Vor dem Einschalten alle Verbindungen mit dem Plan vergleichen.

> **Sicherheit:** Nie mehr als 5 V an einen Pin legen. Nie einen Pin gegen Masse oder 5 V kurzschließen. LEDs immer mit Vorwiderstand. Motoren nicht direkt an einen Pin hängen.